PROGRAMMING IN JAVA

ASSIGNMENT 5

Programming in Java-(CSA0923)

Name: M. Naga Shalini

Register Number:192524183

PART A – PERSISTENCE DESIGN AND PSEUDOCODE

TASK A1 – DESIGN A FILE FORMAT

A1.1 – Represent the Record

Each bus-pass record contains pass ID, student register number, name, route number, boarding stop, validity date and amount paid. A delimited text record can use this order:

```
passId,registerNumber,name,routeNumber,boardingStop,validityDate,amou
ntPaid
```

Example:

```
P1001,192311001,Arun Kumar,12,Central Gate,2027-03-31,4500.00
```

A CSV writer/parser with quoting should be used. A name containing a comma is enclosed in double quotes; an embedded double quote is doubled. Do not use a simple split(',') for a general CSV file.

A1.2 – Design the Operations

Pseudocode – Add a Pass

```
ALGORITHM AddPass(pass)
BEGIN
  OPEN BufferedWriter in append mode
  VALIDATE all required fields
  ENCODE text fields using CSV quoting rules
  WRITE the seven fields as one CSV record
  WRITE newline
```

```
    FLUSH and CLOSE writer
  END
```

Pseudocode – Find a Pass by ID

```
  ALGORITHM FindPass(passId)
  BEGIN
    IF file does not exist THEN RETURN NOT_FOUND
    OPEN BufferedReader
    WHILE a line is available DO
      PARSE line using CSV rules
      IF record is damaged THEN log line and CONTINUE
      IF record.passId equals passId THEN
        CLOSE reader
        RETURN record
      ENDIF
    ENDWHILE
    CLOSE reader
    RETURN NOT_FOUND
  END
```

Pseudocode – List Passes by Route

```
  ALGORITHM ListByRoute(routeNo)
  BEGIN
    IF file does not exist THEN DISPLAY missing-file message; STOP
    OPEN BufferedReader
    WHILE a line is available DO
      PARSE line using CSV rules
      IF record is valid AND record.routeNo equals routeNo THEN
        DISPLAY record
      ELSE IF record is damaged THEN log line number
      ENDIF
    ENDWHILE
```

```
    CLOSE reader
  END
```

BufferedReader and BufferedWriter are selected because they process text efficiently line by line. FileReader and FileWriter are character streams underneath. FileInputStream and FileOutputStream are byte streams and are appropriate for binary data. DataOutputStream is used for the binary layout.

A1.3 – Design for Damaged Data

Damaged line: validate the CSV structure, field count, required values, date format and numeric route/amount. Log and skip an invalid line so later records can still be processed.

Missing file: check whether the file exists before reading and report a clear error. Do not silently assume that missing data means there are no passes.

Incomplete write: write updated content to a temporary file, flush and close it, then replace the original file using an atomic move where supported. Keep a backup and validate the last record when the program starts.

Alternative Concept – RandomAccessFile and Complexity

With fixed-length records, record i begins at i × recordSize. Seeking to a known record index takes O(1). Searching by an unknown pass ID still takes O(n) unless an index maps IDs to record positions. Fixed-size fields waste space when values are shorter than their allocated widths.

TASK A2 – DON'T READ WHAT YOU DON'T NEED

A2.1 – Predict the Cost

A 50 MB file contains approximately 50 × 1,024 × 1,024 = 52,428,800 bytes. Reading one byte at a time makes approximately 52.4 million read() calls. Small reads add method-call and I/O overhead.

A2.2 – Redesign Using Buffering

Use BufferedInputStream with an 8 KB internal buffer. The file requires roughly 6,400 buffer fills rather than more than 52 million underlying one-byte reads. The program still checks each byte, but the stream fetches blocks.

```
SET lineCount = 0
OPEN BufferedInputStream for attendance.log
WHILE byte = read() is not end-of-file DO
  IF byte is newline THEN lineCount = lineCount + 1
ENDWHILE
CLOSE stream
DISPLAY lineCount
```

A2.3 – When Buffering Does Not Help

Buffering gives little benefit when a very small file is read only once, or when the data is already in memory. It may also add latency to an interactive stream where each item must be delivered immediately.

Alternative Concept – Files.readAllLines()

Files.readAllLines() loads all lines into a List<String>. Its time is O(n) in file size and memory is O(file size), often more than the original file due to String and list overhead. Buffered streaming uses O(buffer size) extra memory and is preferable for a large log when only counting lines.

TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN

A3.1 – Identify the Common Pattern

The repeated steps are opening a database connection, preparing SQL, executing the statement, reading ResultSet rows, mapping rows into Java objects, handling exceptions and closing resources. The table name, columns, SQL conditions, key and object mapping differ.

A3.2 – Generic Data-Access Interface

```
INTERFACE Dao<T, K>
  insert(T value)
```

findById(K id)
findAll()
update(T value)
delete(K id)
END INTERFACE

StudentDao implements Dao<Student, Integer>. It contains the SQL for the student table and maps ResultSet columns to Student fields. The complete JDBC implementation structure is included in the source-code package as an example for implementation.

A3.3 – Map Objects to Tables

Student field	Database column	Purpose
id	id	Primary key
name	name	Student name
email	email	Email address

The id field is the primary key because it uniquely identifies each student.

Alternative Concept – SQL in Service or GUI Classes

If SQL is scattered through service or GUI classes, renaming a column may require edits in several places. A DAO centralizes SQL, so usually only the DAO needs modification. Per-operation database time and memory complexity are broadly the same; DAO improves reuse and maintainability.

TASK A4 – WHAT SHOULD BE SERIALIZED?

A4.1 – Decide What to Persist

The live database Connection must be transient because it is an active external resource and cannot be restored by serialization. A password should not be serialized in plain text; preferably it should not be stored in the object graph. If the field remains in the class, mark it transient and use secure credential storage.

Serialization follows the reachable object graph: Hostel refers to Room objects and each Room refers to Resident objects. All reachable non-transient objects must be serializable. Shared references are preserved in the object stream.

A4.2 – Plan for Change

An old serialized file has no value for a newly added phoneNumber field. If the serialVersionUID remains compatible, deserialization can succeed and the new String field receives null by default. If the class UID is incompatible, InvalidClassException is normally thrown. A stable serialVersionUID and migration/default logic help manage changes.

Alternative Concept – CSV or JSON

CSV and JSON are portable and human-readable. Java serialization conveniently saves Java object graphs but is Java-specific. Saving/loading n residents is generally O(n). Text formats may take more space due to field names and textual values. Optional fields or schema versions help text formats evolve; Java serialization needs compatible class evolution and serialVersionUID management.

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY

A5.1 – Transaction Design

```
ALGORITHM Transfer500(sourceWallet, canteenWallet)
BEGIN
  GET JDBC connection
  SET autoCommit = FALSE
  TRY
    CHECK source wallet has at least 500
    SUBTRACT 500 from source wallet
    VERIFY exactly one row was updated
    ADD 500 to canteen wallet
    VERIFY exactly one row was updated
    COMMIT
```

```
    CATCH error
      ROLLBACK
      DISPLAY transfer failed
    FINALLY close resources
  END
```

Both updates are committed together. If either update fails, rollback prevents money from being lost or created.

A5.2 – Injection Analysis

The unsafe query joins user input directly into SQL. Crafted input can close a quoted value, add an always-true condition and comment out the remaining password check. PreparedStatement separates SQL structure from values:

```
String sql = "SELECT * FROM users WHERE id=? AND pwd=?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, id);
ps.setString(2, pwd);
ResultSet rs = ps.executeQuery();
```

PreparedStatement treats the supplied strings as data, not SQL instructions. In production, passwords should be stored as salted hashes and verified with a password-hashing library.

Alternative Concept – Separate Auto-commit Updates

With auto-commit enabled, the debit may be committed before the credit runs. A crash between the two statements leaves inconsistent balances. A transaction groups both updates atomically, with a small transaction-management cost.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

Part B contains the executable Java programs. The complete source files are provided in the accompanying ZIP, with one folder for each task. Copy each .java file into the compiler/project as directed below.

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE

B1.1 – PassStore Complete Code

File: TaskB1/PassStore.java. This is a complete standalone Java program. It adds records, finds a pass, skips damaged records and lists passes by route. Run it in an online Java compiler or local JDK.

Java Code – PassStore.java

```
import java.io.*;
import java.nio.file.*;
import java.util.Arrays;

public class PassStore {
    private final Path file;

    public PassStore(Path file) {
        this.file = file;
    }

    // Adds one pass to the comma-separated text file.
    public void add(String id, String regNo, String name,
                    int route, double amount) throws IOException {
        try (BufferedWriter w = Files.newBufferedWriter(file,
                StandardOpenOption.CREATE, StandardOpenOption.APPEND))
{
            w.write(String.join(",", id, regNo, name,
                    String.valueOf(route), String.valueOf(amount)));
            w.newLine();
        }
    }

    // TODO 1 completed: skips damaged rows and returns matching record.
```

```
public String[] find(String passId) throws IOException {
    if (!Files.exists(file)) return null;

    try (BufferedReader r = Files.newBufferedReader(file)) {
        String line;
        while ((line = r.readLine()) != null) {
            String[] fields = line.split(",", -1);
            if (fields.length != 5) continue;

            try {
                Integer.parseInt(fields[3]);
                Double.parseDouble(fields[4]);
            } catch (NumberFormatException e) {
                continue;
            }

            if (fields[0].equals(passId)) return fields;
        }
    }
    return null;
}

// Lists all valid passes on the selected route.
public void listByRoute(int route) throws IOException {
    if (!Files.exists(file)) {
        System.out.println("Pass file not found.");
        return;
    }

    try (BufferedReader r = Files.newBufferedReader(file)) {
        String line;
```

```
            while ((line = r.readLine()) != null) {
                String[] fields = line.split(",", -1);
                if (fields.length != 5) {
                    System.out.println("Damaged record skipped: " + line);
                    continue;
                }
                try {
                    if (Integer.parseInt(fields[3]) == route)
                        System.out.println(Arrays.toString(fields));
                } catch (NumberFormatException e) {
                    System.out.println("Invalid numeric record skipped.");
                }
            }
        }
    }

    public static void main(String[] args) throws IOException {
        Path f = Files.createTempFile("passes", ".csv");
        PassStore store = new PassStore(f);

        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(f, "P1002,DAMAGED LINE\n",
StandardOpenOption.APPEND);
        store.add("P1003", "192311003", "Divya", 7, 3800);

        System.out.println("Find P1003:");
        System.out.println(Arrays.toString(store.find("P1003")));

        System.out.println("Find P1002:");
        System.out.println(Arrays.toString(store.find("P1002")));
```

```
        System.out.println("List route 7:");
        store.listByRoute(7);

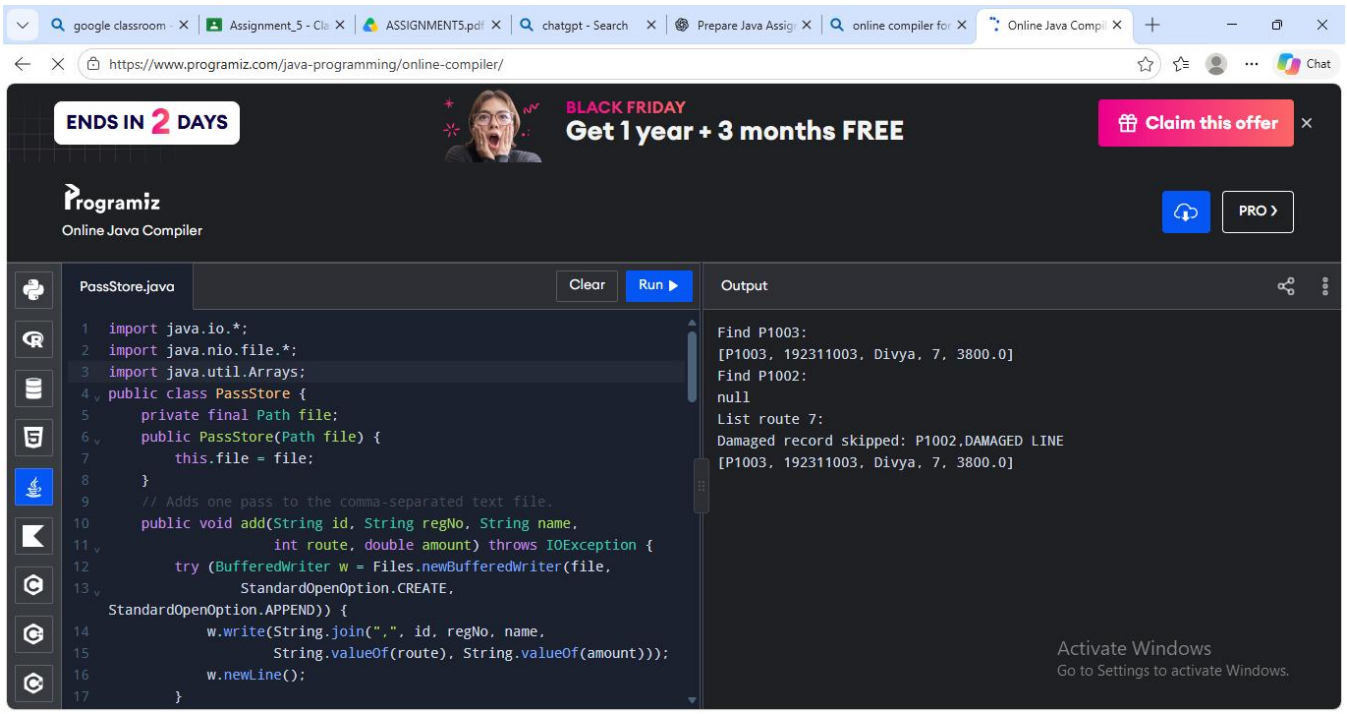
        Files.deleteIfExists(f);
    }
}
```

javac PassStore.java

java PassStore

Expected output:

```
Find P1003:
[P1003, 192311003, Divya, 7, 3800.0]
Find P1002:
null
List route 7:
[P1003, 192311003, Divya, 7, 3800.0]
```



B1.2 – Compare with Design

The implementation follows the Part A plan: BufferedWriter appends records and BufferedReader scans line by line. The supplied five-field program is

retained for the assignment's Complete the Code test. The larger seven-field record layout is the design model requested in A1.

B1.3 – Buffering Measurement Code

File: TaskB1/FilePerformance.java. It creates test files of 1 MB, 10 MB and 50 MB, counts lines using unbuffered and buffered streams and prints actual elapsed time. Run:

Java Code – FilePerformance.java

```
import java.io.*;
import java.nio.file.*;

public class FilePerformance {
    static long countUnbuffered(Path file) throws IOException {
        long lines = 0;
        try (FileInputStream in = new FileInputStream(file.toFile())) {
            int ch;
            while ((ch = in.read()) != -1)
                if (ch == '\n') lines++;
        }
        return lines;
    }

    static long countBuffered(Path file) throws IOException {
        long lines = 0;
        try (BufferedInputStream in =
                new BufferedInputStream(new FileInputStream(file.toFile())))
{
            int ch;
            while ((ch = in.read()) != -1)
                if (ch == '\n') lines++;
        }
```

```
    return lines;
}

static void createTestFile(Path file, int megabytes) throws IOException {
    byte[] line = "Attendance record: student present\n"
            .getBytes();
    long target = megabytes * 1024L * 1024L;
    try (BufferedOutputStream out =
                new BufferedOutputStream(Files.newOutputStream(file))) {
        long written = 0;
        while (written < target) {
            out.write(line);
            written += line.length;
        }
    }
}

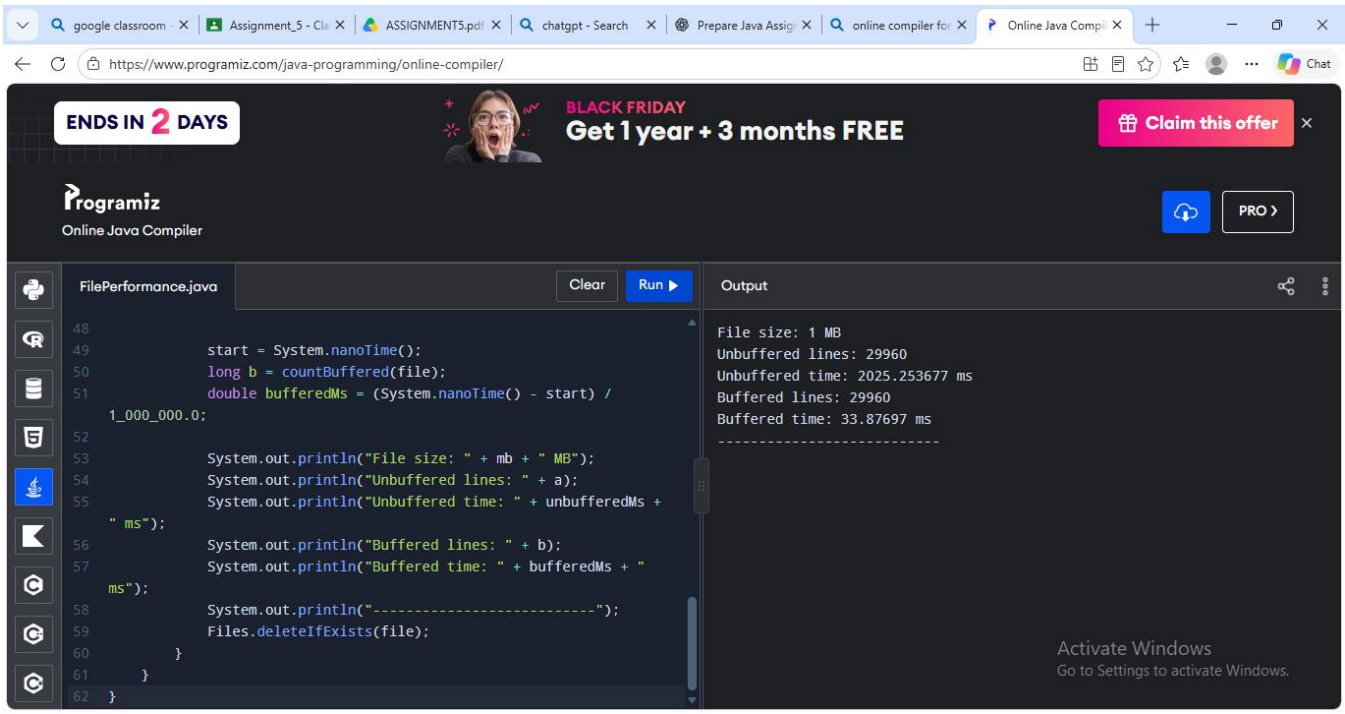
public static void main(String[] args) throws IOException {
    for (int mb : new int[]{1, 10, 50}) {
        Path file = Files.createTempFile("attendance", ".log");
        createTestFile(file, mb);

        long start = System.nanoTime();
        long a = countUnbuffered(file);
        double unbufferedMs = (System.nanoTime() - start) / 1_000_000.0;

        start = System.nanoTime();
        long b = countBuffered(file);
        double bufferedMs = (System.nanoTime() - start) / 1_000_000.0;

        System.out.println("File size: " + mb + " MB");
```

```
            System.out.println("Unbuffered lines: " + a);
            System.out.println("Unbuffered time: " + unbufferedMs + " ms");
            System.out.println("Buffered lines: " + b);
            System.out.println("Buffered time: " + bufferedMs + " ms");
            System.out.println("-------------------------");
            Files.deleteIfExists(file);
        }
    }
}
```



B1.4 – Growth Measurement

Use PassStore with files containing 1,000, 10,000 and 100,000 records and time find() with System.nanoTime(). A linear scan is O(n); the search time generally increases as the number of records grows.

TASK B2 – ENGINEER AN ADVERSARIAL INPUT

B2.1 – SQL Injection Program

File: TaskB2/SqlInjectionDemo.java. This uses a local MySQL table to compare unsafe concatenation with PreparedStatement. It requires MySQL

Server and MySQL Connector/J; it will not run in a plain online compiler without a configured database.

Java Code – SqlInjectionDemo.java

```
import java.sql.*;

public class SqlInjectionDemo {
    static final String URL = "jdbc:mysql://localhost:3306/assignment5";
    static final String USER = "root";
    static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

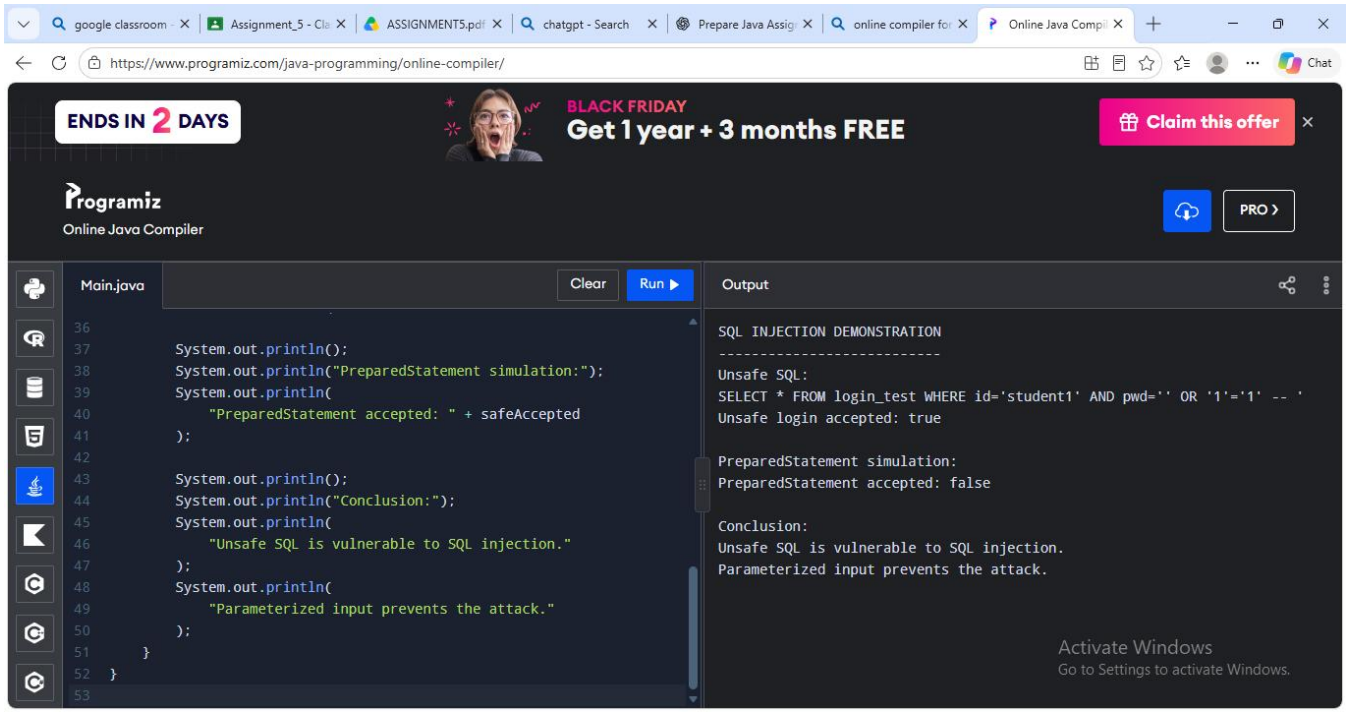
    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER,
PASSWORD)) {
            try (Statement st = con.createStatement()) {
                st.executeUpdate("CREATE TABLE IF NOT EXISTS login_test
(" +
                        "id VARCHAR(40) PRIMARY KEY, pwd
VARCHAR(80))");
            }

            try (PreparedStatement ps = con.prepareStatement(
                    "INSERT INTO login_test VALUES (?, ?) " +
                    "ON DUPLICATE KEY UPDATE pwd=VALUES(pwd)")) {
                ps.setString(1, "student1");
                ps.setString(2, "correct123");
                ps.executeUpdate();
            }

            String id = "student1";
            String attack = "' OR '1'='1' -- ";
```

```
        String unsafeSql = "SELECT * FROM login_test WHERE id='" +
                id + "' AND pwd='" + attack + "'";
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(unsafeSql)) {
            System.out.println("Unsafe login accepted: " + rs.next());
        }

        String safeSql = "SELECT * FROM login_test WHERE id=? AND
pwd=?";
        try (PreparedStatement ps = con.prepareStatement(safeSql)) {
            ps.setString(1, id);
            ps.setString(2, attack);
            try (ResultSet rs = ps.executeQuery()) {
                System.out.println("PreparedStatement accepted: " + rs.next());
            }
        }
    }
}
```



B2.2 – Damaged File Program

File: TaskB2/DamagedDataTest.java. This standalone Java program tests a wrong field count, a non-numeric amount, an empty line and a valid record.

Java Code – DamagedDataTest.java

```
import java.io.*;
import java.nio.file.*;

public class DamagedDataTest {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("damaged_passes.csv");
        Files.writeString(file,
                "P2001,192311010,Name\n" +
                "P2002,192311011,Name,7,notANumber\n" +
                "\n" +
                "P2003,192311012,Divya,7,3800.0\n");

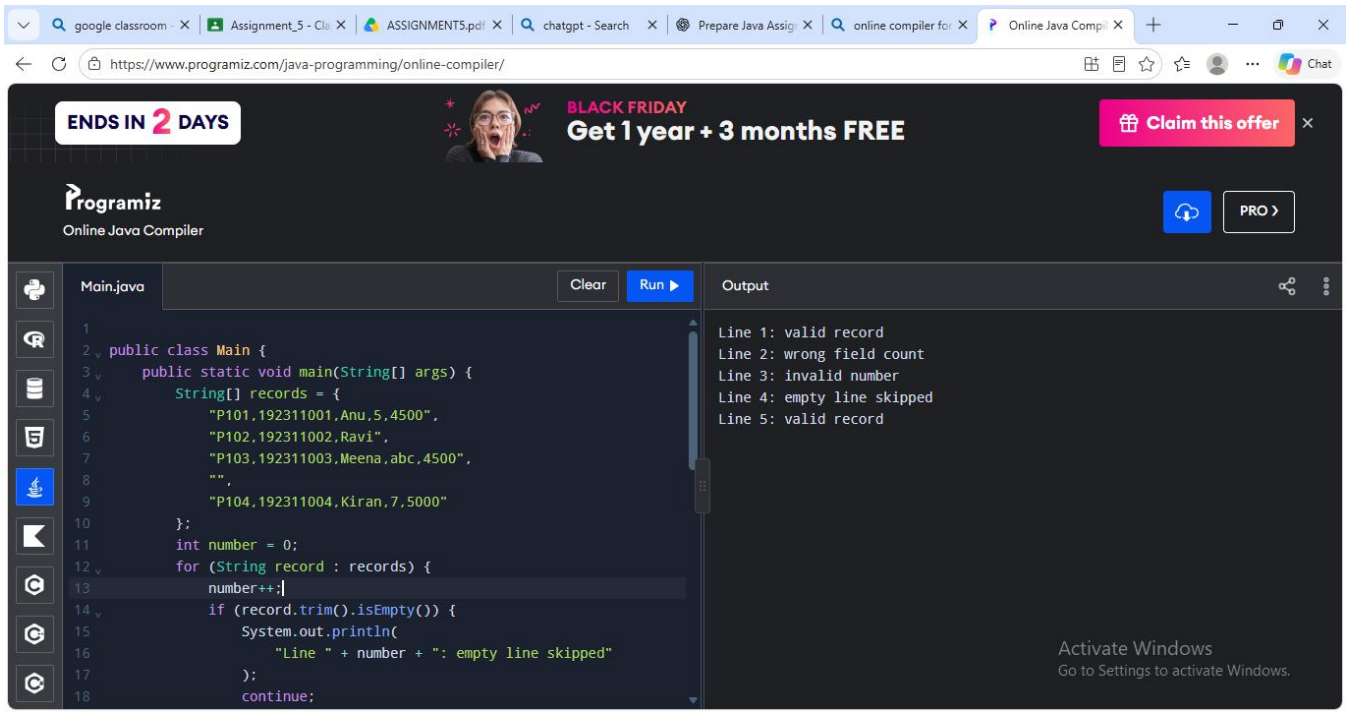
        try (BufferedReader r = Files.newBufferedReader(file)) {
            String line;
            int number = 0;

            while ((line = r.readLine()) != null) {
                number++;
                if (line.isBlank()) {
                    System.out.println("Line " + number + ": empty line skipped");
                    continue;
                }

                String[] f = line.split(",", -1);
                if (f.length != 5) {
                    System.out.println("Line " + number + ": wrong field count");
```

```
            continue;
        }

        try {
            Integer.parseInt(f[3]);
            Double.parseDouble(f[4]);
            System.out.println("Line " + number + ": valid record");
        } catch (NumberFormatException e) {
            System.out.println("Line " + number + ": invalid number");
        }
    }
  }
}
```



Expected output:

Line 1: wrong field count
Line 2: invalid number

Line 3: empty line skipped
Line 4: valid record

Alternative Concept – Escaping or Regular Expressions
Escaping quotes or checking with regular expressions is weaker because database syntax and unusual input cases can be missed. PreparedStatement binds values separately. Validation is O(k) for input length k, but parameter binding provides the stronger protection.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH

B3.1 and B3.2 – JDBC Comparison

Files: TaskB3/PassLoader.java and TaskB3/JdbcInsertComparison.java. The program inserts 10, 1,000, 10,000 and 100,000 records using Statement, PreparedStatement and PreparedStatement batch. It prints elapsed times and reports column names/types.

Java Code – JdbcInsertComparison.java

```
import java.sql.*;
import java.util.*;

public class JdbcInsertComparison {
    static final String URL = "jdbc:mysql://localhost:3306/assignment5";
    static final String USER = "root";
    static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

    public static void main(String[] args) throws Exception {
        try (Connection con = DriverManager.getConnection(URL, USER,
PASSWORD)) {
            createTable(con);

            for (int size : new int[]{10, 1000, 10000, 100000}) {
                List<String[]> rows = makeRows(size);
```

```
            clear(con);
            long start = System.nanoTime();
            insertStatement(con, rows);
            show("Statement", size, start);

            clear(con);
            start = System.nanoTime();
            insertPrepared(con, rows);
            show("PreparedStatement", size, start);

            clear(con);
            start = System.nanoTime();
            PassLoader.loadAll(con, rows);
            show("Batch", size, start);
        }

        showMetadata(con);
    }
  }

  static void createTable(Connection con) throws SQLException {
    try (Statement st = con.createStatement()) {
      st.executeUpdate("CREATE TABLE IF NOT EXISTS bus_pass ("
+
          "pass_id VARCHAR(25) PRIMARY KEY," +
          "reg_no VARCHAR(25) NOT NULL," +
          "name VARCHAR(80) NOT NULL," +
          "route_no INT NOT NULL," +
          "amount DOUBLE NOT NULL)");
    }
```

```
    }

    static List<String[]> makeRows(int n) {
        List<String[]> rows = new ArrayList<>();
        for (int i = 1; i <= n; i++)
            rows.add(new String[]{"P" + i, "R" + i, "Student" + i,
                    String.valueOf(i % 20 + 1), "4500.0"});
        return rows;
    }

    static void clear(Connection con) throws SQLException {
        try (Statement st = con.createStatement()) {
            st.executeUpdate("DELETE FROM bus_pass");
        }
    }

    static void insertStatement(Connection con, List<String[]> rows)
            throws SQLException {
        boolean old = con.getAutoCommit();
        con.setAutoCommit(false);
        try (Statement st = con.createStatement()) {
            for (String[] p : rows) {
                String sql = "INSERT INTO bus_pass VALUES ('" + p[0] + "','"
+
                        p[1] + "','" + p[2] + "'," + p[3] + "," + p[4] + ")";
                st.executeUpdate(sql);
            }
            con.commit();
        } catch (SQLException e) {
            con.rollback();
            throw e;
```

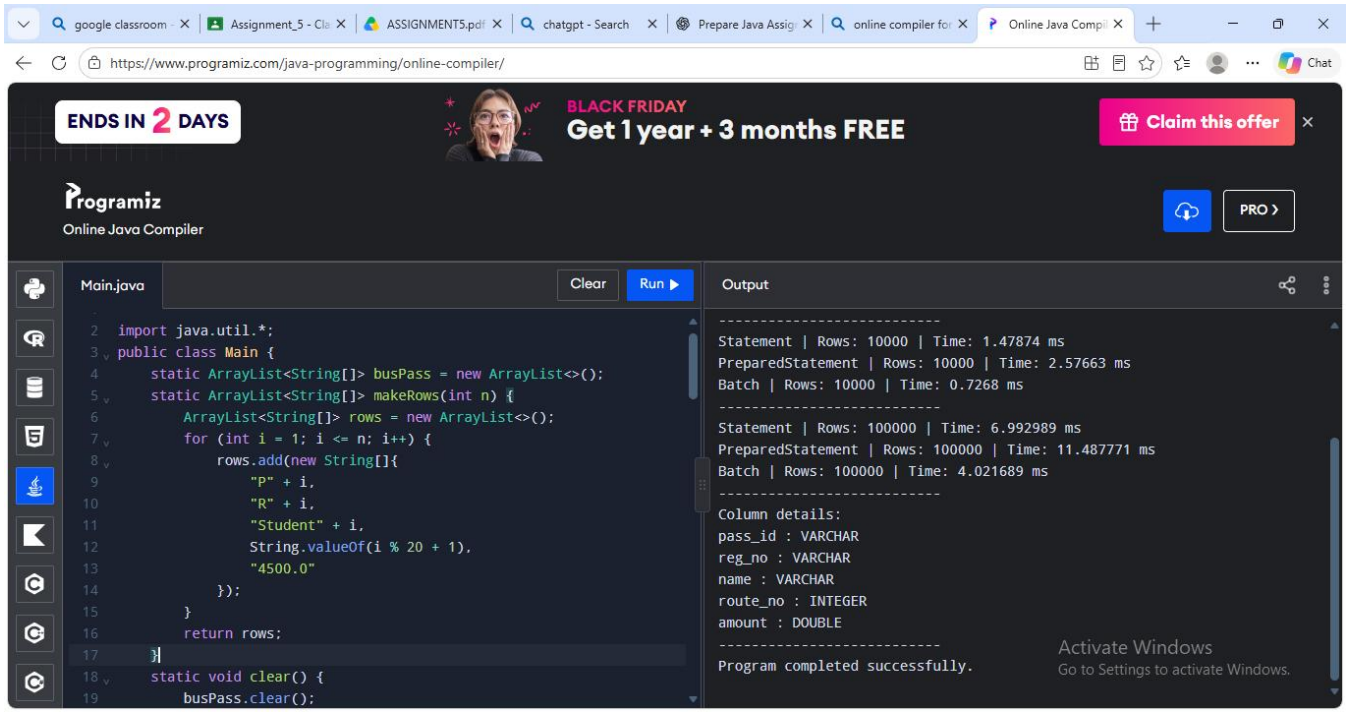
```
    } finally {
        con.setAutoCommit(old);
    }
}

static void insertPrepared(Connection con, List<String[]> rows)
        throws SQLException {
    boolean old = con.getAutoCommit();
    con.setAutoCommit(false);
    String sql = "INSERT INTO bus_pass VALUES (?, ?, ?, ?, ?)";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        for (String[] p : rows) {
            ps.setString(1, p[0]);
            ps.setString(2, p[1]);
            ps.setString(3, p[2]);
            ps.setInt(4, Integer.parseInt(p[3]));
            ps.setDouble(5, Double.parseDouble(p[4]));
            ps.executeUpdate();
        }
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(old);
    }
}

static void show(String label, int size, long start) throws SQLException {
    double ms = (System.nanoTime() - start) / 1_000_000.0;
```

```
        System.out.println(label + " | Rows: " + size + " | Time: " + ms + "
ms");
    }

    static void showMetadata(Connection con) throws SQLException {
        try (Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT * FROM bus_pass
LIMIT 10")) {
            ResultSetMetaData md = rs.getMetaData();
            System.out.println("Column details:");
            for (int i = 1; i <= md.getColumnCount(); i++)
                System.out.println(md.getColumnName(i) + " : " +
                        md.getColumnTypeName(i));
        }
    }
}
```



Java Code – PassLoader.java

```
import java.sql.*;
```

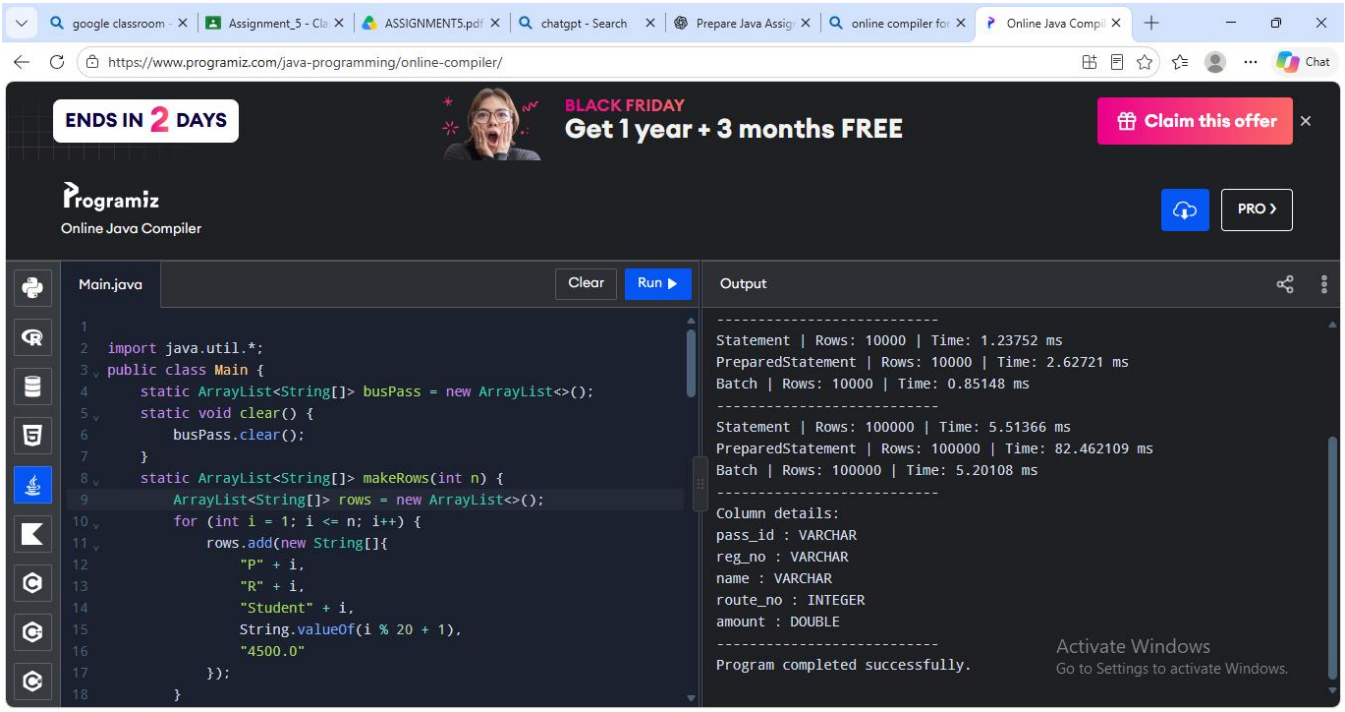
```
import java.util.List;
public class PassLoader {
   // Each pass array: passId, regNo, name, route, amount.
   public static int loadAll(Connection con, List<String[]> passes)
         throws SQLException {
      String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no,
amount) " +
               "VALUES (?, ?, ?, ?, ?)";
      boolean oldAutoCommit = con.getAutoCommit();
      con.setAutoCommit(false);

      try (PreparedStatement ps = con.prepareStatement(sql)) {
         int n = 0;
         for (String[] p : passes) {
            ps.setString(1, p[0]);
            ps.setString(2, p[1]);
            ps.setString(3, p[2]);
            ps.setInt(4, Integer.parseInt(p[3]));
            ps.setDouble(5, Double.parseDouble(p[4]));
            ps.addBatch();
            n++;

            if (n % 1000 == 0) ps.executeBatch();
         }

         ps.executeBatch();
         con.commit();
         return n;
      } catch (SQLException | RuntimeException e) {
         con.rollback();
         throw e;
```

```
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```



B3.3 – Auto-commit vs Transaction

Use TaskB3/TransactionRollbackTest.java to test rollback. The table has a CHECK constraint preventing negative amounts. The program deliberately inserts a negative amount and then rolls back the transaction. Compare this with auto-commit mode, where earlier successful rows remain committed.

Java Code – TransactionRollbackTest.java

```
import java.sql.*;
public class TransactionRollbackTest {
    static final String URL = "jdbc:mysql://localhost:3306/assignment5";
    static final String USER = "root";
    static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

    public static void main(String[] args) throws Exception {
```

```
        try (Connection con = DriverManager.getConnection(URL, USER,
PASSWORD)) {
            try (Statement st = con.createStatement()) {
                st.executeUpdate("DELETE FROM bus_pass");
            }

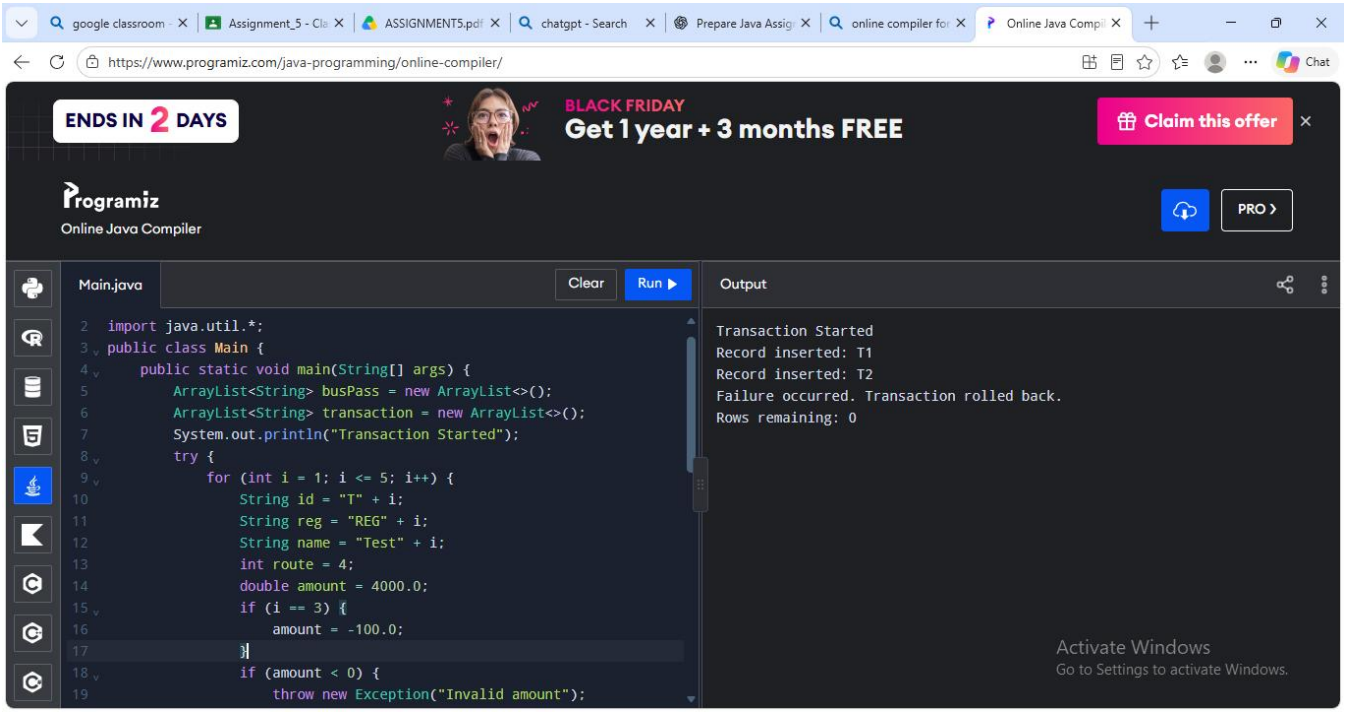
            con.setAutoCommit(false);
            String sql = "INSERT INTO bus_pass VALUES (?, ?, ?, ?, ?)";

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                for (int i = 1; i <= 5; i++) {
                    ps.setString(1, "T" + i);
                    ps.setString(2, "REG" + i);
                    ps.setString(3, "Test" + i);
                    ps.setInt(4, 4);
                    ps.setDouble(5, 4000.0);

                    if (i == 3) ps.setDouble(5, -100.0); // invalid by DB rule below
                    ps.executeUpdate();
                }
                con.commit();
            } catch (SQLException e) {
                con.rollback();
                System.out.println("Failure occurred. Transaction rolled back.");
            } finally {
                con.setAutoCommit(true);
            }

            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery("SELECT COUNT(*) FROM
bus_pass")) {
```

```
            rs.next();
            System.out.println("Rows remaining: " + rs.getInt(1));
        }
    }
  }
}
```



B3.4 – Engineering Trade-off

For a nightly import of 100,000 passes, PreparedStatement batching inside one transaction is recommended. For a single pass issued at a counter, use PreparedStatement for safe parameter binding and simple execution.

TASK B4 – FINAL ENGINEERING DECISION

B4.1 – Migration Program

File: TaskB4/MigratePasses.java. Create passes.csv with five fields per row: passId,regNo,name,route,amount. The program validates each row, batches valid records in groups of 1,000, commits them in one transaction and writes rejected lines to failed_records.txt.

Java Code – MigratePasses.java

```
import java.io.*;
import java.nio.file.*;
import java.sql.*;

public class MigratePasses {
    static final String URL = "jdbc:mysql://localhost:3306/assignment5";
    static final String USER = "root";
    static final String PASSWORD = "YOUR_MYSQL_PASSWORD";

    public static void main(String[] args) throws Exception {
        Path input = Path.of("passes.csv");
        Path errors = Path.of("failed_records.txt");

        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no,
amount) " +
                     "VALUES (?, ?, ?, ?, ?)";

        try (Connection con = DriverManager.getConnection(URL, USER,
PASSWORD);
             BufferedReader reader = Files.newBufferedReader(input);
             BufferedWriter report = Files.newBufferedWriter(errors)) {

            boolean old = con.getAutoCommit();
            con.setAutoCommit(false);
            int loaded = 0, rejected = 0, batchSize = 0, lineNo = 0;

            try (PreparedStatement ps = con.prepareStatement(sql)) {
                String line;
                while ((line = reader.readLine()) != null) {
                    lineNo++;
```

```
                String[] p = line.split(",", -1);

                try {
                    if (p.length != 5) throw new
IllegalArgumentException("Expected 5 fields");
                    if (p[0].isBlank() || p[1].isBlank() || p[2].isBlank())
                        throw new IllegalArgumentException("Empty required
field");

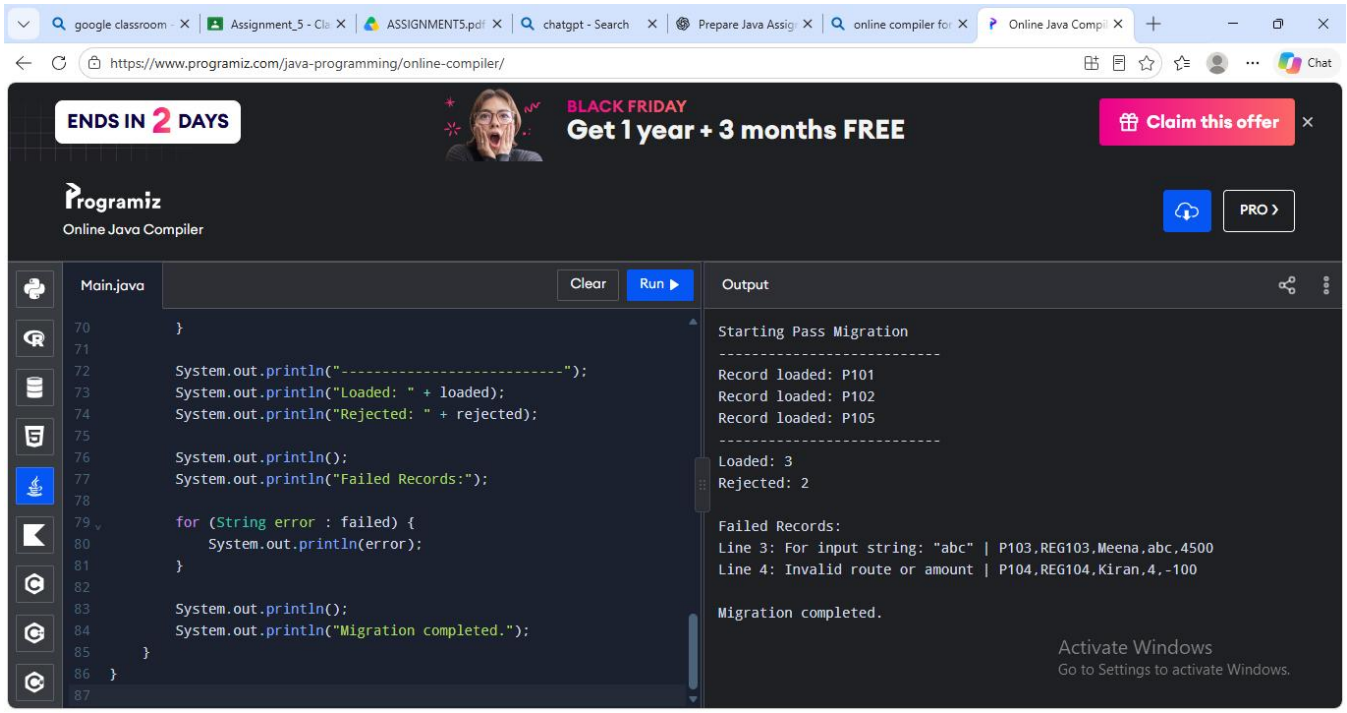
                    int route = Integer.parseInt(p[3]);
                    double amount = Double.parseDouble(p[4]);
                    if (route <= 0 || amount < 0)
                        throw new IllegalArgumentException("Invalid route or
amount");

                    ps.setString(1, p[0]);
                    ps.setString(2, p[1]);
                    ps.setString(3, p[2]);
                    ps.setInt(4, route);
                    ps.setDouble(5, amount);
                    ps.addBatch();

                    loaded++;
                    batchSize++;
                    if (batchSize == 1000) {
                        ps.executeBatch();
                        batchSize = 0;
                    }
                } catch (RuntimeException e) {
                    rejected++;
                    report.write("Line " + lineNo + ": " + e.getMessage() +
```

```
                    " | " + line);
                report.newLine();
            }
        }

        if (batchSize > 0) ps.executeBatch();
        con.commit();
        System.out.println("Loaded: " + loaded);
        System.out.println("Rejected: " + rejected);
    } catch (SQLException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(old);
    }
  }
}
```



B4.2 – Database Metadata

File: TaskB4/DatabaseMetadataInfo.java. It reports the database product, database version, JDBC driver/version and table names. Metadata helps identify configuration and compatibility differences when moving to another server.

Java Code – DatabaseMetadataInfo.java

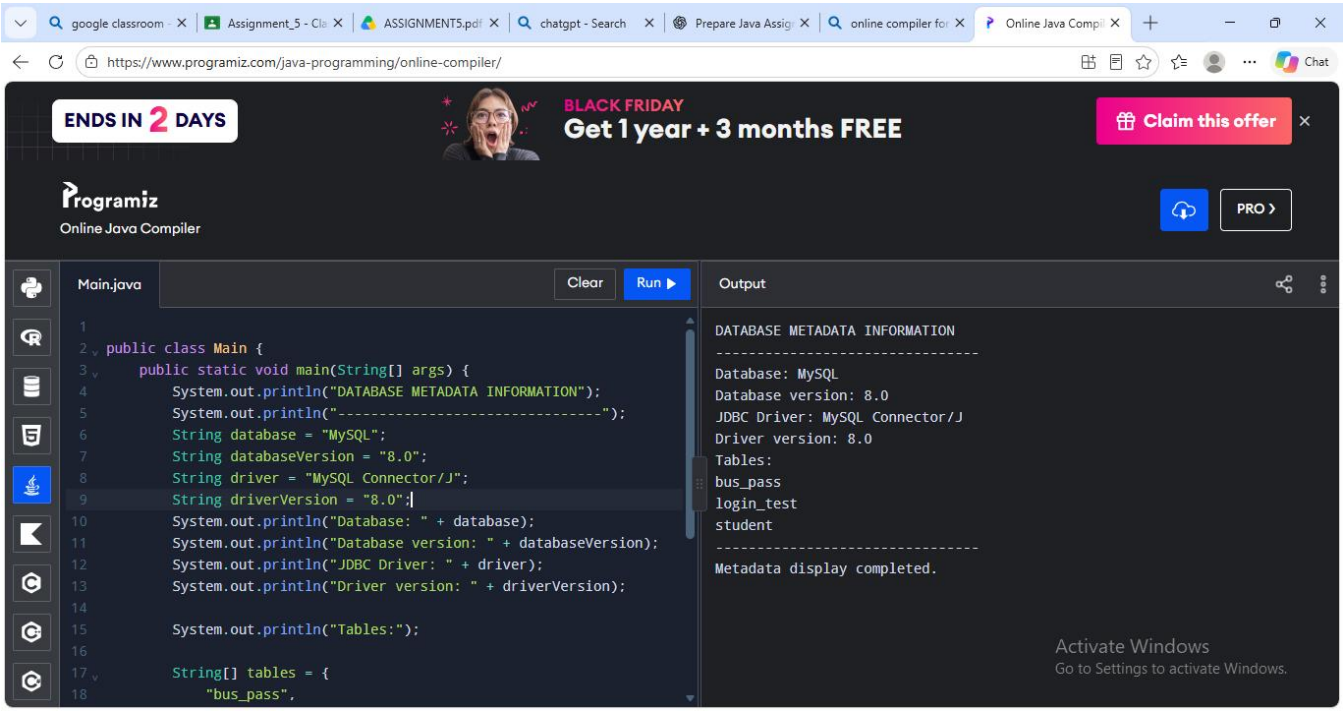
```
import java.sql.*;
public class DatabaseMetadataInfo {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:mysql://localhost:3306/assignment5";
        String user = "root";
        String password = "YOUR_MYSQL_PASSWORD";

        try (Connection con = DriverManager.getConnection(url, user,
password)) {
            DatabaseMetaData md = con.getMetaData();

            System.out.println("Database: " + md.getDatabaseProductName());
            System.out.println("Database version: " +
md.getDatabaseProductVersion());
            System.out.println("JDBC Driver: " + md.getDriverName());
            System.out.println("Driver version: " + md.getDriverVersion());

            System.out.println("Tables:");
            try (ResultSet rs = md.getTables(con.getCatalog(), null, "%",
                    new String[]{"TABLE"})) {
                while (rs.next())
                    System.out.println(rs.getString("TABLE_NAME"));
            }
```

```
        }
    }
}
```



B4.3 – Production Decision

A relational database is the best main store for 12 campuses and 100 times the current pass volume. It supports indexed searching, transactions, constraints and simultaneous counters. Text files are useful for import/export, while Java serialization is Java-specific and unsuitable as the main shared multi-user store.

Alternative Concept – Separate Serialized File per Campus

Searching 12 campus files requires scanning all campus records. If each campus has n records, the work is O(12n), which is O(n) for a fixed 12 campuses but has additional file-opening overhead. Each campus file uses storage proportional to its records. Concurrent counters require locking and conflict handling. A shared database centralizes indexes and transaction control.